



Introducción a JavaScript

(I) Conceptos básicos

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

**Carlos Arévalo, Daniel Ayala, José Calderón,
Margarita Cruz, Inma Hernández, David Ruiz**

UNIVERSIDAD DE SEVILLA

Objetivos



Contenidos

Introducción a ECMAScript y JavaScript

Sintaxis JavaScript

Clases y funciones predefinidas

Browser Object Model

Depuración JS



JavaScript



Brendan Eich en 1995



Se convirtió en estándar ECMA en 1997 (nombre oficial: ECMA-262)

ECMA

European Computer Manufacturers Association: organización que define estándares tecnológicos relacionados con los sistemas de información y comunicación

- <https://www.ecma-international.org/>

Otros ejemplos de estándar ECMA

- Sistemas NFC
- Office Open XML formats
- JSON
- Lenguajes de programación: C#, Eiffel,
- ... (https://en.wikipedia.org/wiki/List_of_Ecma_standards)

Características

Lenguaje interpretado (no compilado)

Tipado dinámico

JavaScript es un lenguaje de programación como cualquier otro y puede utilizarse en diversos contextos.

Sin embargo, donde su uso está realmente extendido es en la web, donde se utiliza para dar dinamismo en las páginas web.

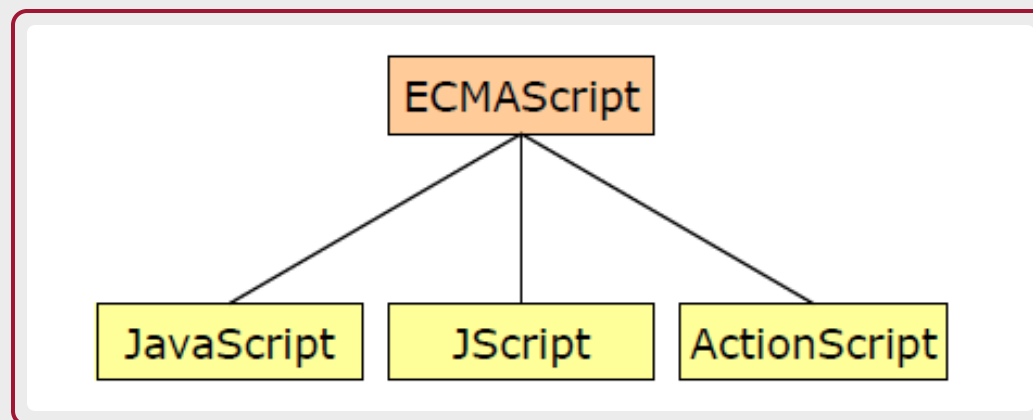
No sólo se utiliza en frontend, sino también en backend para programar software de servidores

- NodeJS , Express , MeteorJS

ECMAScript vs JavaScript

JavaScript es un lenguaje conforme con la especificación ECMAScript

- Es la implementación más popular de ECMAScript, pero no es la única



Las características principales de JavaScript se basan en el estándar ECMAScript, pero JavaScript tiene también otras características que no están en el estándar.

Cada navegador tiene su propia implementación de la interfaz ECMAScript

Timeline ECMAScript

1995: Sun Microsystems y Netscape anuncian JavaScript.

1997: Se propuso la primera versión del `ECMA-262` (`ECMAScript 1`).

1999: `ECMAScript 3`. Se añaden expresiones regulares y `try/catch`

2009: `ECMAScript 5`. Se añade el soporte para `JSON`, funciones de iteración de arrays, y `use strict`

2015: `ECMAScript 6`: Se añaden `let / const`, valores por defecto para los parámetros

Después de Junio 2015, el nombre `ECMAScript 6` pasó a ser `ECMAScript 2015` (`ES2015`), para llevar un control de los lanzamientos anuales del estándar.

2017: `ECMAScript 2017`: Se añaden propiedades de `Object`, `Shared Memory` y funciones `Async`

Enero 2019: `ECMAScript 2019` (`ES2019`) o `ES10`.

Junio 2020: `ECMAScript 2020` o `ES11`: `Nullish coalescence operator`, `BigInt`

...

Junio 2023: `ECMAScript 2023`: Métodos nuevos para `Array`

Julio 2024: `ECMAScript 2024`: Método `groupBy()` para `Object` y `Map`, métodos para comprobar si una cadena está bien formada

Junio 2025: `ECMAScript 2025`: Nuevos métodos para iteradores, operaciones con conjuntos, mejoras en expresiones regulares.

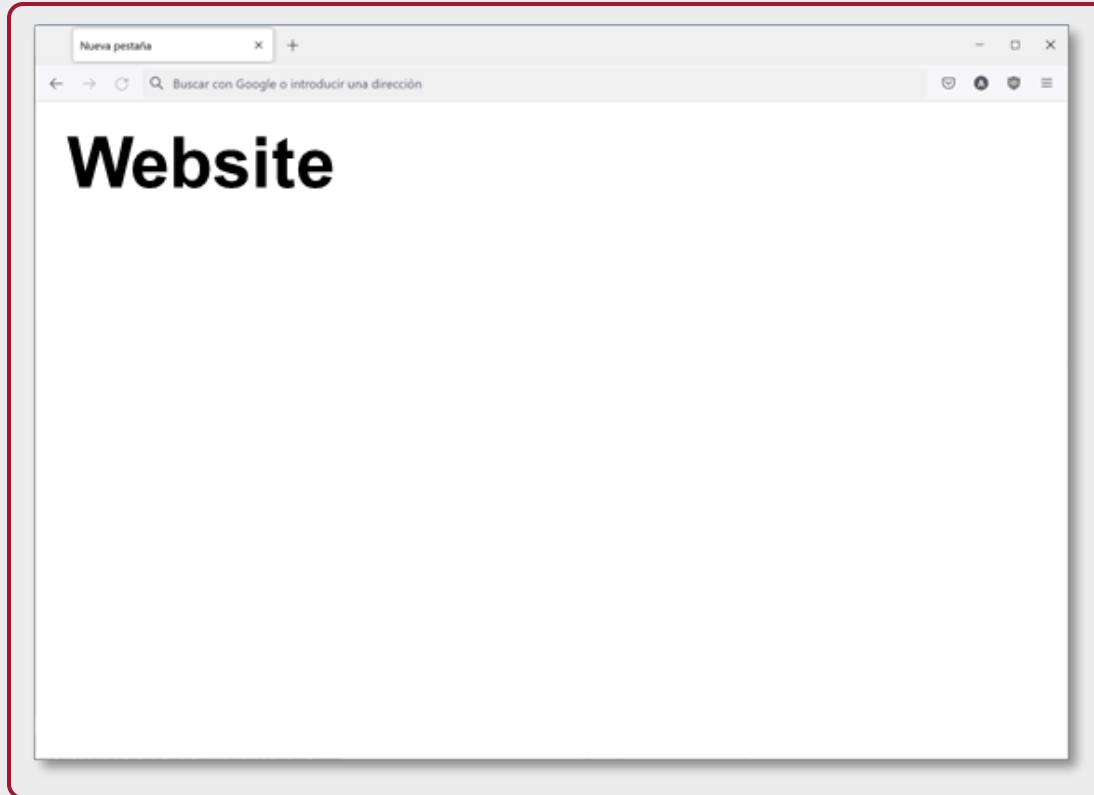
Java vs JavaScript

Solo comparten el nombre y una sintaxis similar
JavaScript inicialmente se denominó Mocha, después LiveScript y finalmente JavaScript (coincidiendo con el momento en que Netscape empezó a soportar Java en su navegador).

- La versión de Internet Explorer se llamó JScript, dado que JavaScript era un nombre registrado

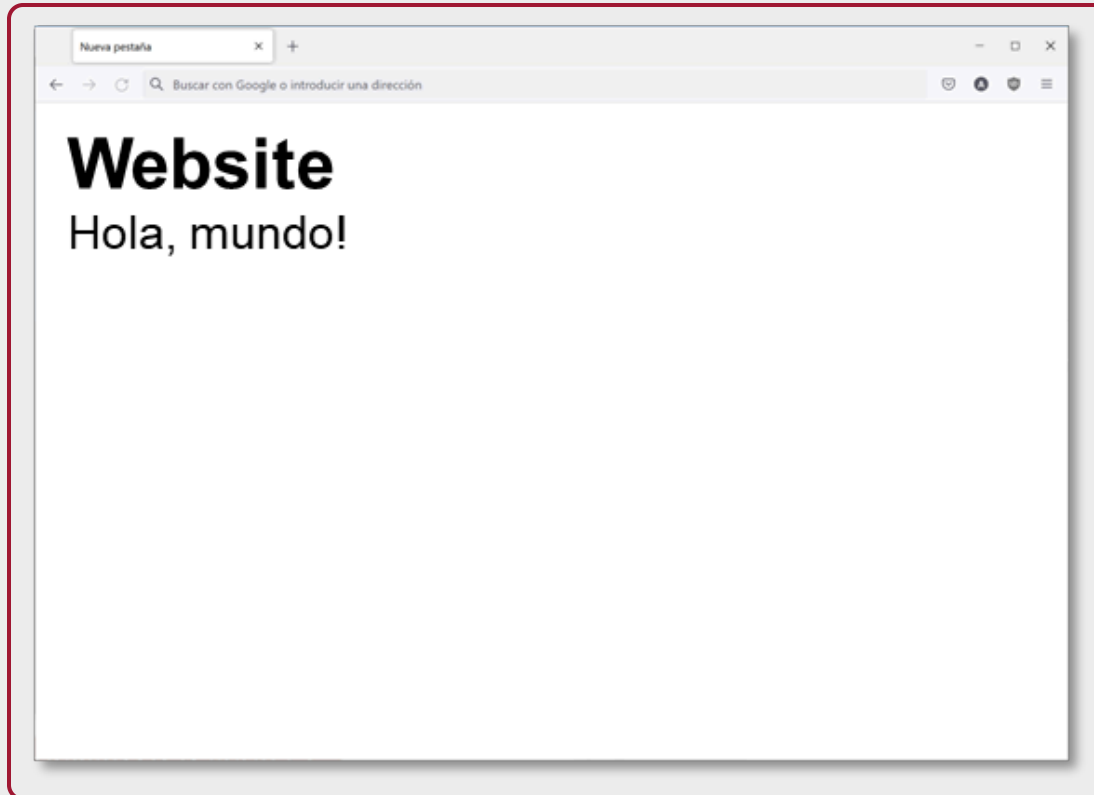


Contexto de JavaScript



```
<html>
  <head>...</head>
  <body>
    <h1>Website</h1>
  </body>
</html>
```

Contexto de JavaScript



```
<html>
  <head>...</head>
  <body>
    <h1>Website</h1>
    <script>
      document.write("¡Hola, mundo!");
    </script>
  </body>
</html>
```

Código JavaScript

Ubicación

- En el documento HTML
 - Dentro del elemento `head`
 - Dentro del elemento `body`
 - Como gestor de evento `onEvento="código"`
- En un archivo separado
 - Fichero `.js`
- Antes, el código JS que se incluía directamente en el documento HTML debía ir entre comentarios HTML por motivos de compatibilidad.
 - Hoy en día no es necesario, salvo que se necesite compatibilidad con navegadores muy antiguos

Orden de ejecución del código

Dentro de un documento HTML, pueden aparecer fragmentos de código JavaScript tanto en la cabecera como en distintas partes del cuerpo.

El código JavaScript se ejecuta siempre en el orden en el que aparece en el código HTML.

Lo habitual dentro del elemento `head` es poner sólo declaraciones de variables y funciones.

Script dentro del documento (body)

```
<html>
  <head>
    <title>Título del documento</title>
    <!-- otra información de cabecera -->
  </head>
  <body>
    <!-- contenido del documento -->
    <!-- más contenido del documento -->
    <script>
      // Código JavaScript
    </script>
  </body>
</html>
```


Script dentro del documento (head)

```
<html>
  <head>
    <title>Título del documento</title>
    <!-- otra información de cabecera -->
    <script>
      // Código JavaScript
    </script>
  </head>
  <body>
    <!-- contenido del documento -->
  </body>
</html>
```

Script en un fichero separado (.js)

```
<html>
  <head>
    <title>Título del documento</title>
    <!-- otra información de cabecera -->
    <script src="micodigo.js"></script>
  </head>
  <body>
    <!-- contenido del documento -->
  </body>
</html>
```

```
// Fichero micodigo.js

// Código JS
```

Si usamos HTML5, no es necesario usar el atributo type pues, por defecto, se considera que el código es JavaScript

Contenidos

Introducción a ECMAScript y JavaScript

Sintaxis JavaScript

Clases y funciones predefinidas

Browser Object Model

Depuración JS

Variables

Se pueden definir con `var` o `let`

- Las variables definidas con `var` tienen ámbito de función (o global si se declaran fuera de una función). Se pueden redefinir en el mismo ámbito donde se declaran.
- Las variables definidas con `let` tienen el ámbito del bloque donde son declaradas (función, `if`, `for`, objeto...). No se pueden redefinir en el mismo ámbito donde se declaran.

En código JS moderno se recomienda usar `let`, ya que tiene un comportamiento más parecido a otros lenguajes.

Se pueden declarar constantes con `const`.

```
var age = 24;  
let empName = "Juan";
```

Variables

JavaScript es sensible a las mayúsculas y las minúsculas, por lo que una variable nombre no es la misma que otra Nombre.

No es obligatorio declarar las variables, pero se recomienda.

- Ojo, con el modo estricto sí es obligatorio

```
let nPract = 10, nTheory = 9;  
let nTotal = (nPract * 0.6) + (nTheory * 0.4);
```



Variables

Ámbito (scope): Determina desde dónde son accesibles las variables que se definen

Tipos de ámbito

- **Global:** Variable definida fuera de una función; es accesible desde todo el código/script de una página web
- **Local:** Variable definida dentro de una función; es accesible desde la función donde se define
- **De bloque:** Variable definida con `let` dentro de un bloque; es accesible desde el bloque

Local vs Global

Local

```
// aquí NO se puede usar empName  
function myFunction() {  
  let empName = "Juan";  
  // aquí se puede usar empName  
}
```

Global

```
let empName = "Juan";  
// aquí SÍ se puede usar empName  
function myFunction() {  
  let empName = "Juan";  
  // aquí también se puede usar empName  
}
```

Bloque

```
// aquí NO se puede usar empName  
function myFunction() {  
  {  
    let empName = "Juan";  
  
    // aquí se puede usar empName  
  }  
  // aquí NO se puede usar empName  
}
```


Comentarios

```
// Esto es un comentario de una línea  
  
/* Esto es un comentario  
... de varias líneas */
```

Flujo condicional

if-else

```
if (condition) {  
    doSomething();  
} else if (anotherCondition) {  
    doSomethingElse();  
} else {  
    doAnotherThing();  
}
```

switch

```
switch (x) {  
    case 1:  
        xIs1();  
        break;  
    case 2:  
        xIs2();  
    default:  
        xIsOther();  
}
```

Operador ternario

```
let x = (number % 2 === 0) ? "Even" : "Odd";
```

Bucles

for

```
for (let i = 0; i < array.length; i++) {  
  let elem = array[i];  
  ...  
}
```

while

```
while (condition) {  
  ...  
}
```

for...of

```
for (let elem of array) {  
  ...  
}
```

do-while

```
do {  
  ...  
} while (condition);
```

Funciones

Notación estándar

```
function foo(arg1, arg2, arg3) {  
  // Do something...  
}
```

Notación flecha

```
let foo = (arg1, arg2, arg3) => {  
  // Do something...  
};
```

La notación flecha se suele utilizar para funciones anónimas o de una línea, ya que queda más conciso:

```
function add(x, y) {  
  return x + y;  
}  
// vs  
(x, y) => x + y;
```

¡Las funciones también son variables y se pueden referenciar por su nombre!

Contenidos

Introducción a ECMAScript y JavaScript

Sintaxis JavaScript

Clases y funciones predefinidas

Browser Object Model

Depuración JS

Tipos básicos

Primitivos: Solo contienen un valor a la vez

- Cadenas: `"¡Hola, mundo!"`, `'epi@barriosesame.es'`
- Números: `12`, `22.4`, `-5`
- Booleans: `true`, `false`

Compuestos: Contienen colecciones de valores y tipos más complejos de datos

- `Object`
- `Array`
- `Function`

Nulos:

- `undefined`
- `null`

Cadenas (I)

Se definen como secuencias de caracteres entre comillas simples (' '), dobles (" ") o acentos graves (` `)

```
let a = 'Comillas simples';  
let b = "Comillas dobles";
```

La secuencia puede incluir a su vez comillas simples o dobles, siempre que no cierren las comillas abiertas previamente (si lo hacen, hay que escaparlas):

```
// comilla simple dentro de comillas dobles let a = "Let's have a cup of coffee.";  
let a = "Let's hava a cup of coffee.";
```

```
// comillas dobles dentro de comillas simples  
let b = 'He said "Hello" and left.';
```

```
// Escape de comillas  
let c = 'We\'ll never give up.';
```

Cadenas (II)

Las cadenas definidas con acentos graves (` `) pueden contener comillas simples y dobles:

```
let x = `This is a "string" with both 'types' of quotes`;
```

Las cadenas definidas mediante estos acentos pueden contener variables mediante la sintaxis `${}`

```
let name = "John";  
let age = 30;  
// Interpolación de variables  
let x = `My name is ${name} and I'm ${age} years old`;  
// x = "My name is John and I'm 30 years old"
```


Clase String

Propiedades

- `length` : longitud de la cadena.

Métodos

- `charAt(n)` : devuelve el carácter colocado en la posición `n`
- `match(c)` : dice si la subcadena `c` pertenece a la cadena
- `substring(x,y)` : devuelve la subcadena que va de `x` a `y` (no incluido)
- `substr(x,y)` : devuelve la subcadena que empieza en la posición `x` y tiene `y` caracteres.
- `toLowerCase()` : convierte a mayúsculas
- `toUpperCase()` : convierte a minúsculas
- `replace()` : cambia las ocurrencias de una subcadena por otra
- `concat()` : une dos o más cadenas
- `trim()` : elimina espacios en blanco al principio y al final de la cadena
- `split()` : dado un separador, parte la cadena y la convierte en un array
- `toString()` : convierte un objeto en una cadena

Ejemplos – Funciones de String

```
let str = "Hi, I'm a string";

let thirdCharacter = str.charAt(2);
// thirdCharacter = ","

let substring = str.substring(0, 3);
// substring = "Hi,"

let upper = str.toUpperCase();
// upper = "HI, I'M A STRING"

let lower = str.toLowerCase();
// lower = "hi, i'm a string"

let includes = str.includes("string");
// includes = true

let replaced = str.replace("string", "text");
// replaced = "Hi, I'm a text"
```

Arrays

Colecciones de datos ordenados:

```
let ls = ["Alice", "Bob", "Charlie"];
```

Acceso mediante índices:

```
let x = ls[1]; // x = "Bob"  
ls[2] = "Mary"; // ls = ["Alice", "Bob", "Mary"]
```

Iterables mediante `for...of`

```
for (let name of ls) {  
  console.log(name);  
}
```

Clase Array

Propiedades

- `length` : longitud del array

Métodos

- `join()` : representa todos los elementos en una cadena usando un separador (por defecto, coma)
- `reverse()` : invierte el orden de los elementos
- `sort()` : ordena los elementos del array
- `concat()` : Une los elementos de dos arrays en uno solo
- `filter()` : Devuelve un array que retiene los elementos del array original que cumplen una determinada condición , expresada como una función
- `find()` : Devuelve el primer elemento de un array que cumple una determinada condición, expresada como una función
- `indexOf()` : Busca un elemento en el array y devuelve su posición
- `pop()` : Devuelve y elimina el último elemento de un array
- `push()` : Añade al final del array un elemento
- `slice()` : Devuelve un fragmento del array

Ejemplos – funciones de Array (I)

join() :

```
let fruits = ["Banana", "Orange", "Peach", "Mango"];  
let string = fruits.join();  
// string = "Banana,Orange,Peach,Mango"
```

reverse() :

```
let fruits = ["Banana", "Orange", "Peach", "Mango"];  
fruits.reverse();  
// fruits = ["Mango", "Peach", "Orange", "Banana"]
```

sort() :

```
let fruits = ["Banana", "Orange", "Peach", "Mango"];  
fruits.sort();  
// fruits = ["Banana", "Mango", "Orange", "Peach"]
```

Ejemplos – funciones de Array (II)

`concat()` :

```
let fruits = ["Banana", "Orange", "Peach", "Mango"];
let moreFruits = ["Strawberry", "Pear"];

let allFruits = fruits.concat(moreFruits);
// allFruits = ["Banana", "Orange", "Peach", "Mango", "Strawberry", "Pear"]
```

`filter()` :

```
function isAdult(age) {
  return age >= 18;
}

let ages = [32, 33, 16, 40, 12];
let adults = ages.filter(isAdult);

// adults = [32, 33, 40]
```

Ejemplos – funciones de Array (III)

`find()` :

```
function isAdult(age) {  
    return age >= 18;  
}  
  
let ages = [32, 33, 16, 40];  
let firstAdult = ages.find(isAdult);  
// firstAdult = 32
```

`indexOf()` :

```
let fruits = ["Banana", "Orange", "Peach", "Mango"];  
let posPeach = fruits.indexOf("Peach");  
// posPeach = 2
```

Ejemplos – funciones de Array (IV)

pop() :

```
let fruits = ["Banana", "Orange", "Peach", "Mango"];  
let last = fruits.pop();  
// last = "Mango"  
// fruits = ["Banana", "Orange", "Peach"]
```

push() :

```
let fruits = ["Banana", "Orange", "Peach", "Mango"];  
fruits.push("Kiwi");  
// fruits = ["Banana", "Orange", "Peach", "Mango", "Kiwi"]
```

slice() :

```
let fruits = ["Banana", "Orange", "Peach", "Mango"];  
let someFruits = fruits.slice(1, 3);  
// someFruits = ["Orange", "Peach"]
```


Objects (I)

Conjuntos de atributo-valor, similares a los `Maps` de `Java` o los `Dicts` de `Python` *:

```
let person = { age: 24, name: "John" };  
  
let car = {  
  wheels: 4,  
  manufacturer: "Toyota",  
  model: "GT86",  
};
```

Acceso a los valores:

```
let empName = person.name; // empName = "John"  
  
let key = "wheels";  
let numWheels = car[key]; // numWheels = 4
```

*Los atributos de un `Object` son todos de tipo `String`. Si se necesitan atributos de otro tipo, se debe usar la clase `Map` de JS.

Objects (II)

Los valores de un `object` pueden ser funciones:

```
function sayHi() {  
  console.log("Hi!");  
}  
  
let person = { age: 24, name: "John", greeting: sayHi };  
person.greeting(); // prints "Hi!"
```

Los nombres de los atributos de un `object` se pueden iterar mediante `for...in` *

```
for (let key in person) {  
  console.log(key); // "age", "name", "greeting"  
}
```

***No confundir con `for...of` para iterar sobre elementos de un array**

Referencias constantes

Se definen con la palabra reservada `const` .

Es obligatorio asignarles un valor en su declaración.

Su ámbito es de bloque (igual que las variables `let`)

Se comportan como las variables definidas mediante `let` , excepto que no se pueden reasignar (se quedan con el valor que se les asigna inicialmente en su declaración).

No son inmutables, son referencias constantes a un valor

- Si el valor es un tipo primitivo, ya no se puede cambiar (en ese caso, sí es inmutable)
- Si el valor es un objeto, se pueden cambiar los valores de las propiedades del objeto (no la referencia al objeto)
- Por ejemplo, se puede declarar mediante `const` un array y cambiar los elementos del array, o declarar un objeto y cambiar un atributo

Ejemplos - Const

```
const age = 23;  
age = 55; // ERROR
```

```
const employee = { name: "Juan López", salary: 1500 };  
employee.name = "Ana López"; // Correcto  
employee = { name: "Ana López", salary: 1500 }; // ERROR
```

```
const fruits = ["Banana", "Pear", "Peach"];  
// Modificar un elemento:  
fruits[0] = "Melon"; // Correcto  
// Añadir un elemento:  
fruits.push("Kiwi"); // Correcto
```

Otras clases predefinidas

Number, **Math**, **Date**, ...

Clase Number

En JavaScript, solo hay un tipo numérico, que puede representarse con o sin decimales (también en notación científica)

Propiedades:

- `MAX_VALUE` : Número máximo posible
- `MIN_VALUE` : Número mínimo posible
- `NaN` : Not a Number (p.ej. raíces negativas)

Métodos:

- `isInteger()` : comprueba si el número es entero
- `isNaN()` : comprueba si el número es `NaN`
- `toExponential(n)` : convierte el número a la notación exponencial
- `toPrecision(p)` : formatea un número a una precisión de longitud `p`
- `toString()` : Convierte un número a cadena

Clase Math

Métodos:

- `abs(num)` : función valor absoluto
- `max(x, y)` : devuelve el máximo de `x` e `y`
- `min(x, y)` : devuelve el mínimo de `x` e `y`
- `pow(base, exp)` : potencia
- `random()` : devuelve un número aleatorio en `[0, 1]`
- `round(num)` : redondea al entero más cercano
- `sin(num)` : función seno
- `sqrt(num)` : función raíz cuadrada
- `tan(num)` : función tangente
- ...

Clase Date

Métodos

- `getDate()` : devuelve el día del mes actual como un entero entre 1 y 31
- `getDay()` : devuelve el día de la semana actual como un entero entre 0 y 6 (0 = Domingo)
- `getHours()` : devuelve la hora del día actual como un entero entre 0 y 23 .
- `getMinutes()` : devuelve los minutos de la hora actual como un entero entre 0 y 59 .
- `getMonth()` : devuelve el mes del año actual como un entero entre 0 y 11 .
- `getSeconds()` : devuelve los segundos del minuto actual como un entero entre 0 y 59 .
- `getFullYear()` : devuelve el año actual. NO USAR `getYear()` , deprecado
- `getTime()` : devuelve el tiempo transcurrido en milisegundos desde el 1 de enero de 1970 (epoch) hasta el momento actual
- `getISOString()` : devuelve la hora actual en formato estandarizado (recomendable para comunicación cliente-servidor)

Ejemplo – Date (I)

```
let now = new Date();  
// now = Tue Feb 15 2022 12:47:58 GMT+0100  
  
let day = now.getDate();  
// day = 15  
  
let dayOfTheWeek = now.getDay(); // Entre 0 y 6!  
// dayOfTheWeek = 2 (Martes)  
  
let hour = now.getHours();  
// hour = 12  
  
let minutes = now.getMinutes();  
// minutes = 47  
  
let seconds = now.getSeconds();  
// seconds = 58  
  
let isoString = now.toISOString();  
// isoString = "2022-02-15T11:47:58.082Z"
```

Ejemplo – Date (II)

```
function dayOfTheWeek() {  
  let today = new Date();  
  
  let days = {  
    0: "Sunday",  
    1: "Monday",  
    2: "Tuesday",  
    3: "Wednesday",  
    4: "Thursday",  
    5: "Friday",  
    6: "Saturday",  
  };  
  
  let currentDay = days[today.getDay()];  
  return currentDay;  
}
```

Contenidos

Introducción a ECMAScript y JavaScript

Sintaxis JavaScript

Clases y funciones predefinidas

Browser Object Model

Depuración JS

Browser Object Model (BOM)

Cuando se usa JavaScript en un navegador, están disponibles varios objetos predefinidos:

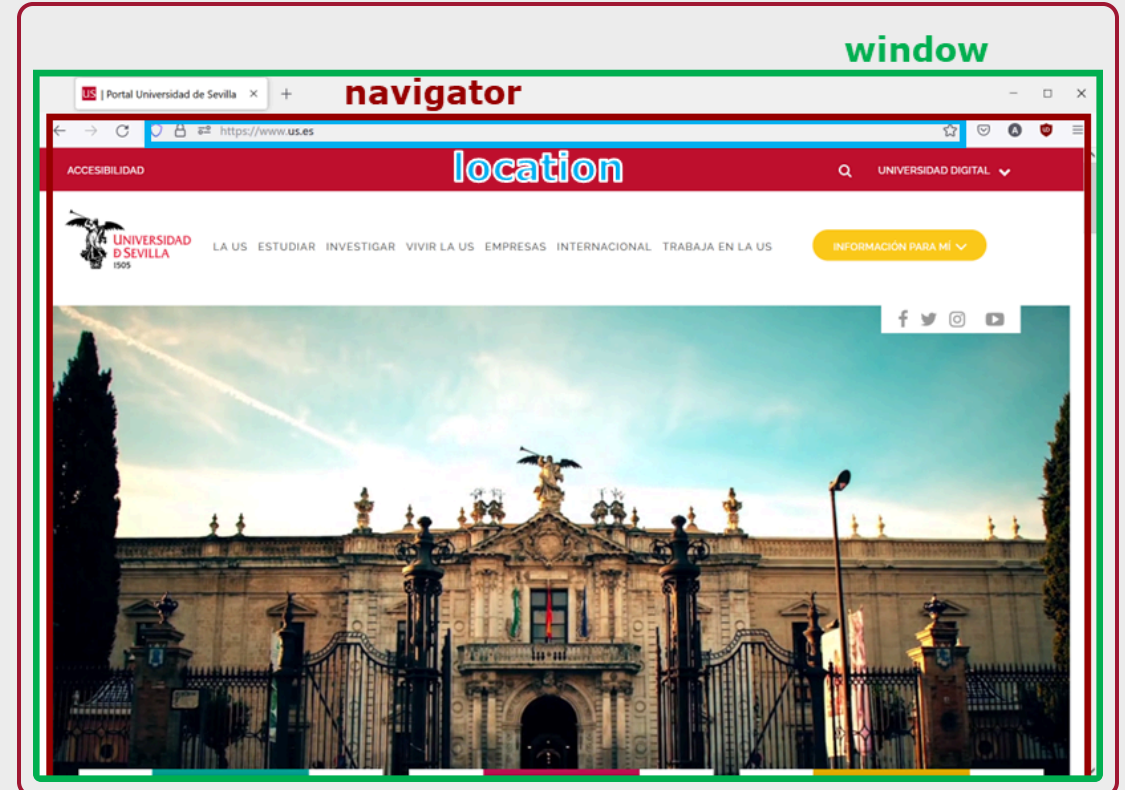
- Algunos están asociados con el propio navegador (BOM – Browser Object Model)
- Otros están relacionados con la estructura del documento HTML (DOM) y son accesibles desde el objeto document:

```
(...)  
  
(...)
```

```
(...)  
document.getElementById("imgmail").src = "mail2.jpg";  
(...)
```

Objetos predefinidos BOM

window : la ventana del navegador.
screen : la pantalla.
navigator : el propio navegador.
history : la historia de navegación.
location : la URL de la página actual.



Objeto `window`

Representa a la ventana del navegador.

Propiedades

- `status` : mensaje de la barra de estado
- `document` : documento HTML visualizado
- `history` : historial de navegación
- `closed` : `true` si la ventana está cerrada
- `length` : número de `frames` que contiene la ventana
- `frames` : un array que contiene todos los `frames` que tiene la ventana, es decir, devuelve un array de objetos de tipo `Frame`

Objeto `window`

Métodos

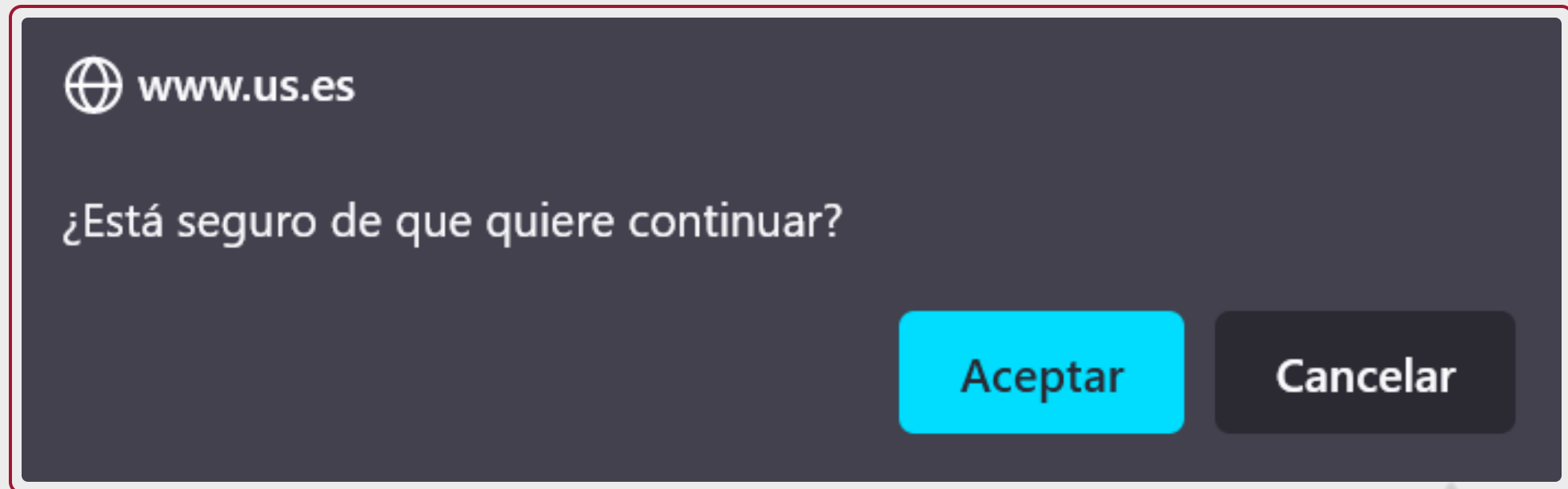
- `open(url)` : abre una nueva ventana del navegador
- `close()` : cierra la ventana
- `alert(msg)` : muestra una ventana con un mensaje



Objeto `window`

Métodos

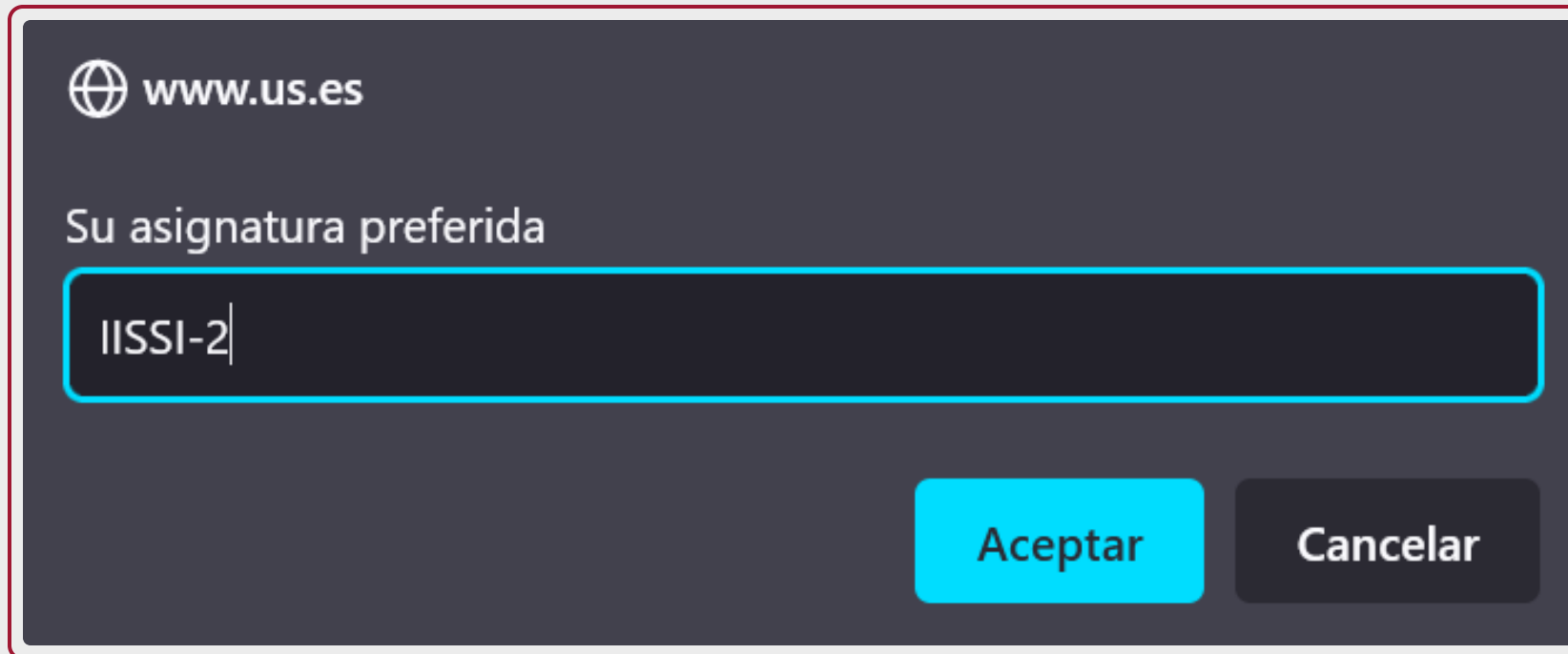
- `confirm(msg)` : muestra una ventana con un botón de cancelar y otro de aceptar; devuelve `true` si se ha pulsado aceptar y `false` si se ha pulsado cancelar



Objeto `window`

Métodos

- `prompt(msg, default)` : muestra una ventana de entrada de datos con un valor por defecto



Objeto navigator

Representa al propio navegador.

Propiedades:

- `appName` : nombre del navegador: `Edge` , `Firefox` , `Chrome` , `Opera` ... en realidad, todos devuelven `Netscape` por compatibilidad antigua
 - No existe una manera fácil de saber cuál es el navegador actual, normalmente se usa `navigator.userAgent` , aunque los navegadores pueden "mentir" por privacidad
- `mimeType` : un array con los tipos que soporta el navegador (`Application` , `Audio` , `Image` , `Message` , `Multipart` , `Text` , `Video` ...)
- `plugins` : un array con los plugins que tiene instalado el navegador
 - Por privacidad, no muestra los que ha instalado el usuario

Objeto `location`

Representa a la URL del documento mostrado en `window`

Propiedades:

- `host` : Contiene el nombre del servidor que ha servido la página y el número de puerto de la URL
- `hostname` : nombre del servidor
- `pathname` : camino sin incluir ni el servidor ni el puerto
- `port` : puerto
- `protocol` : protocolo (`http` , `ftp` ,..)
- `href` : URL que se puede cambiar dinámicamente



Contenidos

Introducción a ECMAScript y JavaScript

Sintaxis JavaScript

Clases y funciones predefinidas

Browser Object Model

Depuración JS

Depuración

Recordatorio: Depurar consiste en encontrar el error que hay tras un fallo en el funcionamiento de un programa

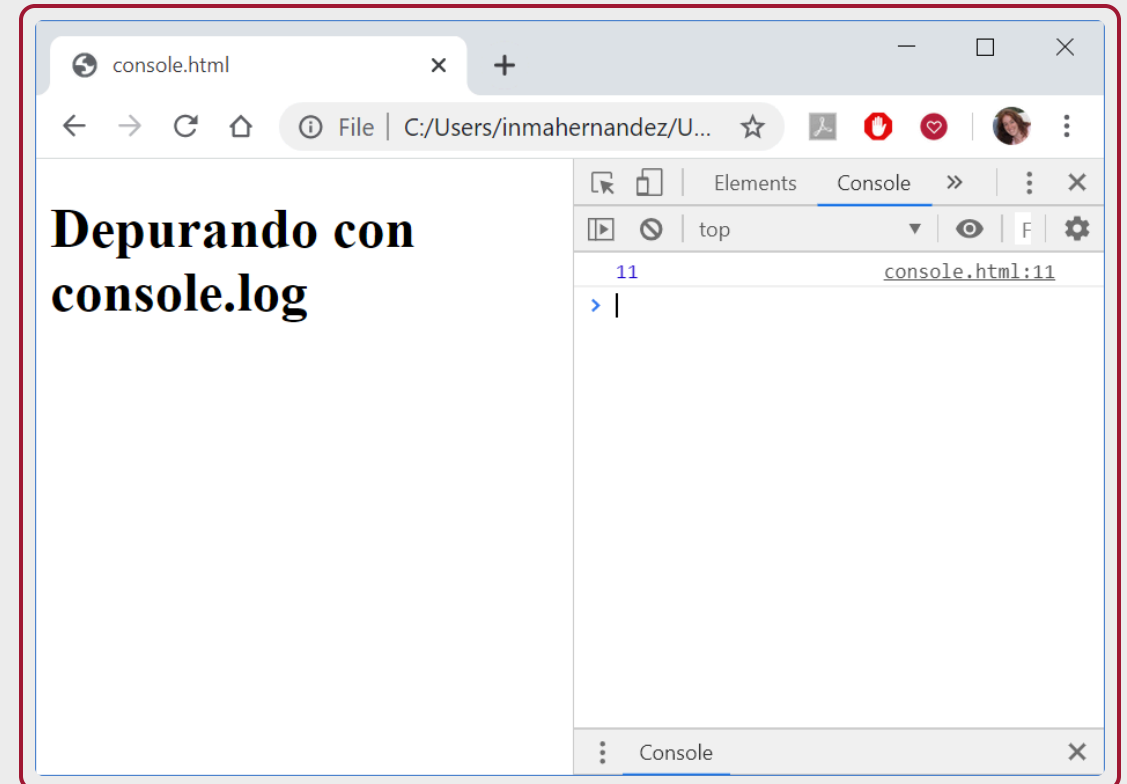
En JavaScript, la depuración es especialmente complicada, dado que muchos errores no se ven en pantalla (solo en la consola)

Todos los navegadores modernos tienen un depurador JavaScript incorporado

- Se pueden activar/desactivar
- Permiten añadir `breakpoints` y examinar el valor de las variables
- Normalmente se activan con `F12` y seleccionando `Console`

console.log

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Depurando con console.log</h1>
    <script>
      a = 5;
      b = 6;
      c = a + b;
      console.log(c);
    </script>
  </body>
</html>
```



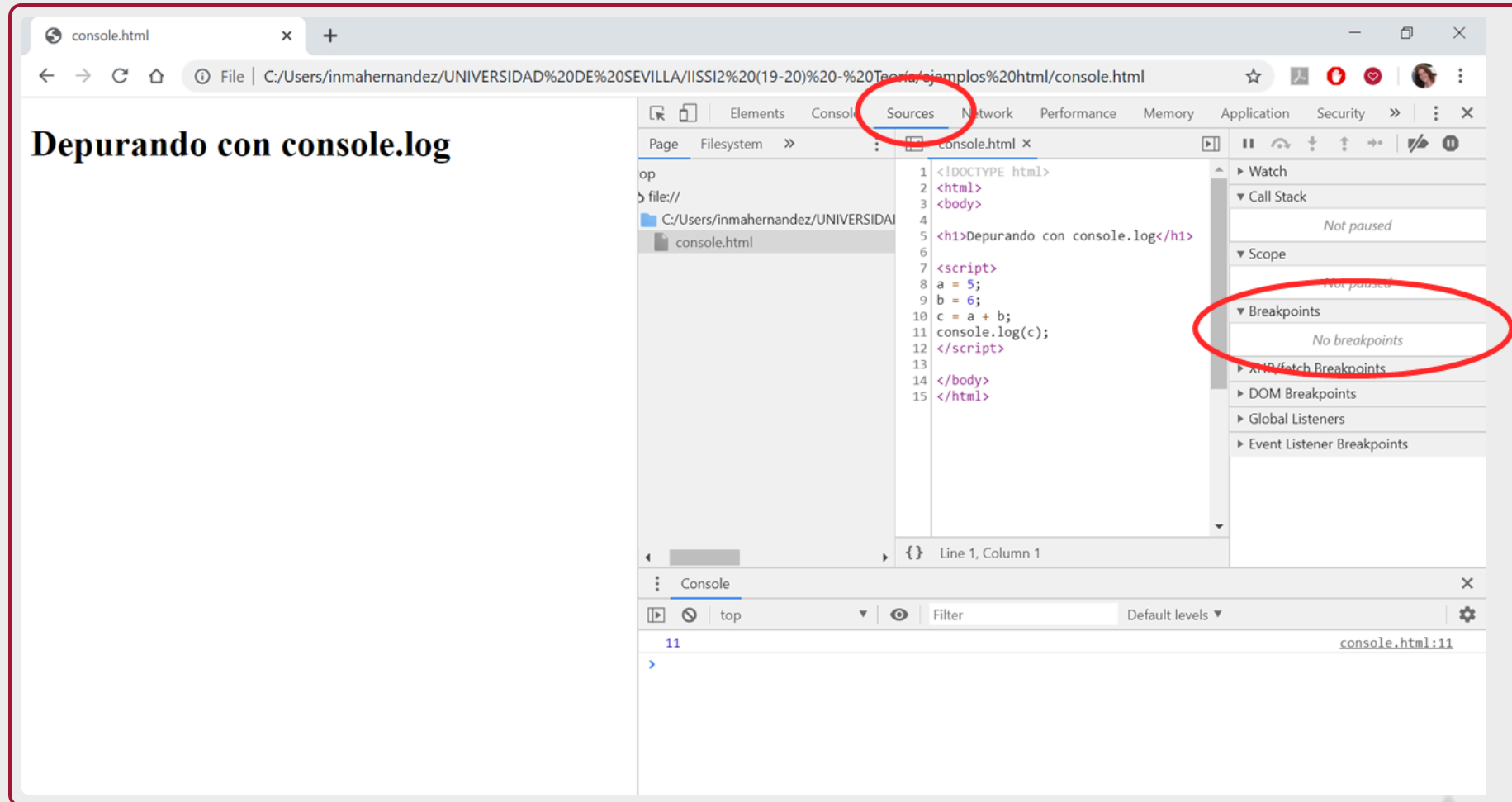
Breakpoints

En la Ventana de depuración, se pueden añadir `breakpoints`

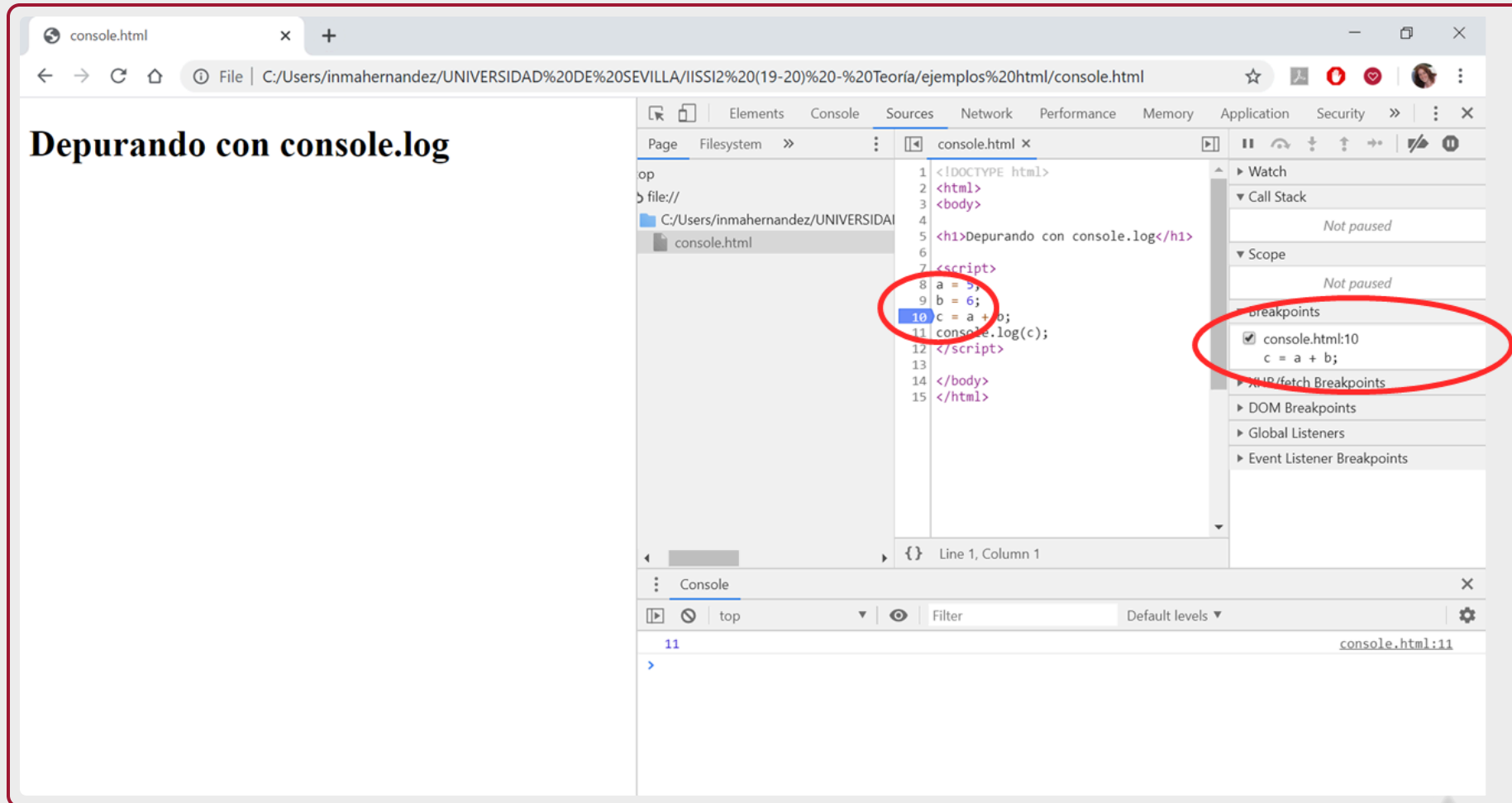
En cada `breakpoint`, JavaScript parará la ejecución y permitirá que el desarrollador examine los valores de las variables.

Después, se puede reanudar la ejecución del código.

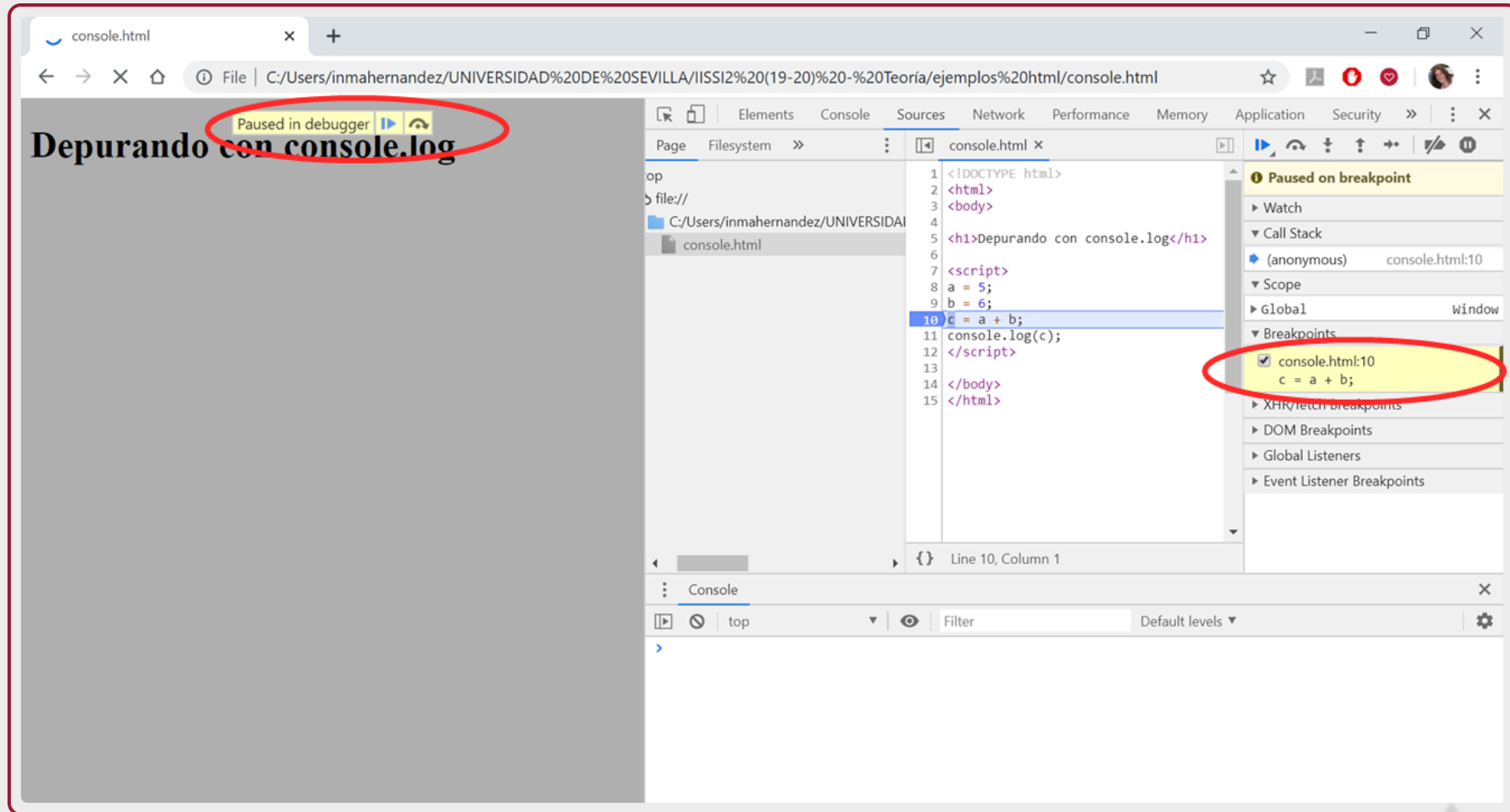
Breakpoints – Ventana de depuración



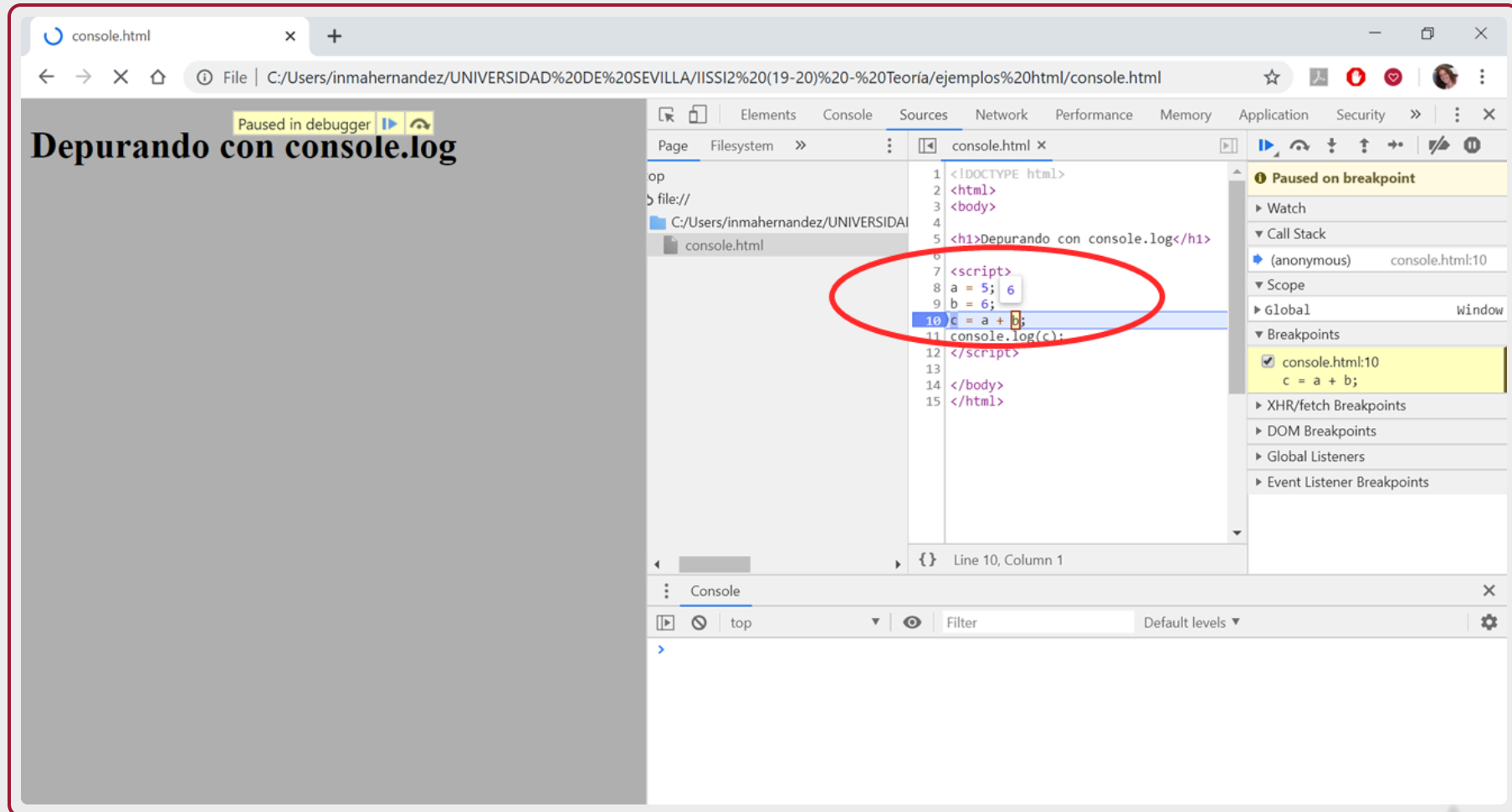
Breakpoints - Definición



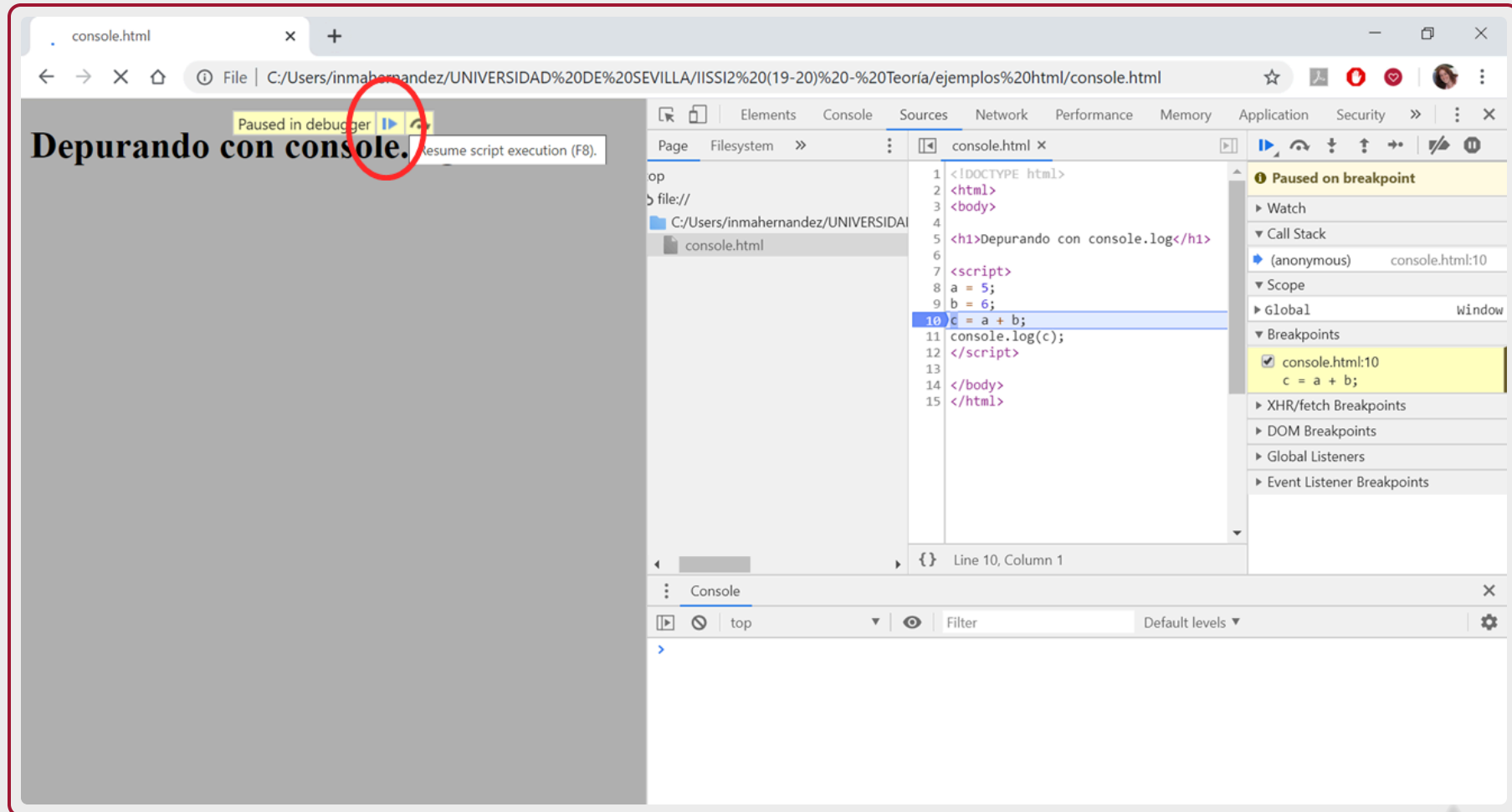
Breakpoints – Alto en la ejecución



Breakpoints – Examinar variables



Breakpoints – Reanudar ejecución



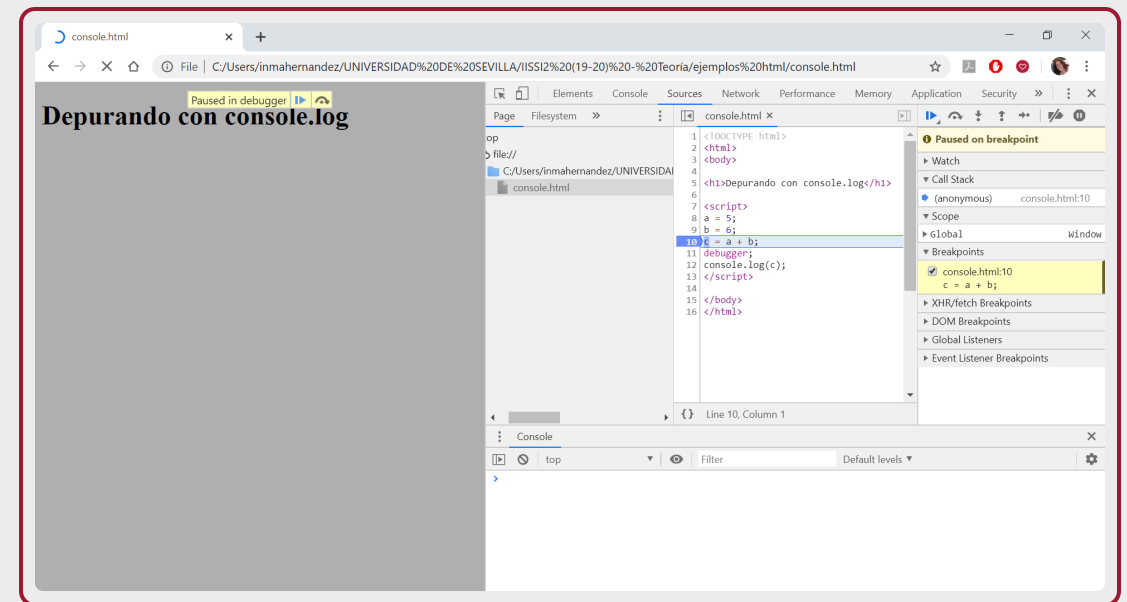
La palabra debugger

La palabra reservada **debugger** detiene la ejecución de JavaScript, y llama a la función de depuración (si está disponible).

Tiene el mismo efecto que definir un **breakpoint** en el depurador.

Si no hay depurador disponible, no tiene efecto.

```
<script>
  a = 5;
  b = 6;
  c = a + b;
  debugger;
  console.log(c);
</script>
```



Referencias

<https://www.w3schools.com/js/>: **tutorial JS**

<https://www.hackerrank.com/domains/tutorials/10-days-of-javascript>: **tutorial de JS en 10 días**

Ejercicios

Entre en HackerRank y resuelva los primeros retos del tutorial en 10 días:

- Day 0: Hello World!
- Day 0: Data Types
- Day 1: Arithmetic operators
- Day 1: Functions
- Day 1: Let and const
- Day 2: Conditional Statements: If-Else
- Day 2: Conditional Statements: Switch
- Day 2: Loops



Conclusiones

- **JavaScript** es el lenguaje de programación más extendido en la **Web** y su evolución está marcada por el estándar **ECMAScript**.
- La **sintaxis básica** de JavaScript se apoya en **variables, ámbito, estructuras de control y funciones**, y conviene usar `let` y `const` en código moderno.
- Los tipos y estructuras más habituales son **cadena**s, **arrays** y **objetos**, que permiten modelar y manipular datos de forma flexible.

Conclusiones

- Las clases predefinidas como **String**, **Array**, **Number**, **Math** y **Date** ofrecen **métodos** y **propiedades** que simplifican muchas operaciones comunes.
- En el navegador, el **BOM** proporciona acceso a objetos como **window**, **navigator** y **location**, que conectan JavaScript con el entorno de ejecución.
- La **depuración** con **console.log**, **breakpoints** y **debugger** es clave para entender el flujo del programa y localizar errores con rapidez.



Gracias

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

UNIVERSIDAD DE SEVILLA